Annex A

Graph Neural Networks: Concepts, Mathematics, and a Crude-Oil Example

A technical primer supporting Chapter 2 of the thesis

A.1 Overview

This annex provides a self-contained technical introduction to the graph neural network (GNN) methods discussed in Chapter 2. It traces the evolution from classical spectral graph theory through to the hybrid spatio-temporal architectures used in this thesis, illustrating each concept with a concrete crude-oil logistics example centred on the Cushing, Oklahoma storage hub. The annex is organised as follows:

- A.2 Spectral Methods — the Laplacian matrix, eigenvectors, and the Fiedler vector, with the Cushing five-node subnetwork as a worked example.
- A.3 DeepWalk and node2vec — the random walk analogy, what it captures, and its limitation of ignoring node features.
- A.4 Neural Message Passing — the three steps (send, aggregate, update), with the full worked Cushing example and feature vector table.
- A.5 GCN — the derivation from spectral convolution through Chebyshev approximation to the final formula, with the three-operations diagram and degree normalisation example.
- A.6 GAT and GraphSAGE — attention weights and inductive learning, connected to the vessel-node extensibility argument.
- A.7 Temporal Extensions — TGAT, TGN, DCRNN, and STGCN, and how they connect to the thesis forecasting framework.

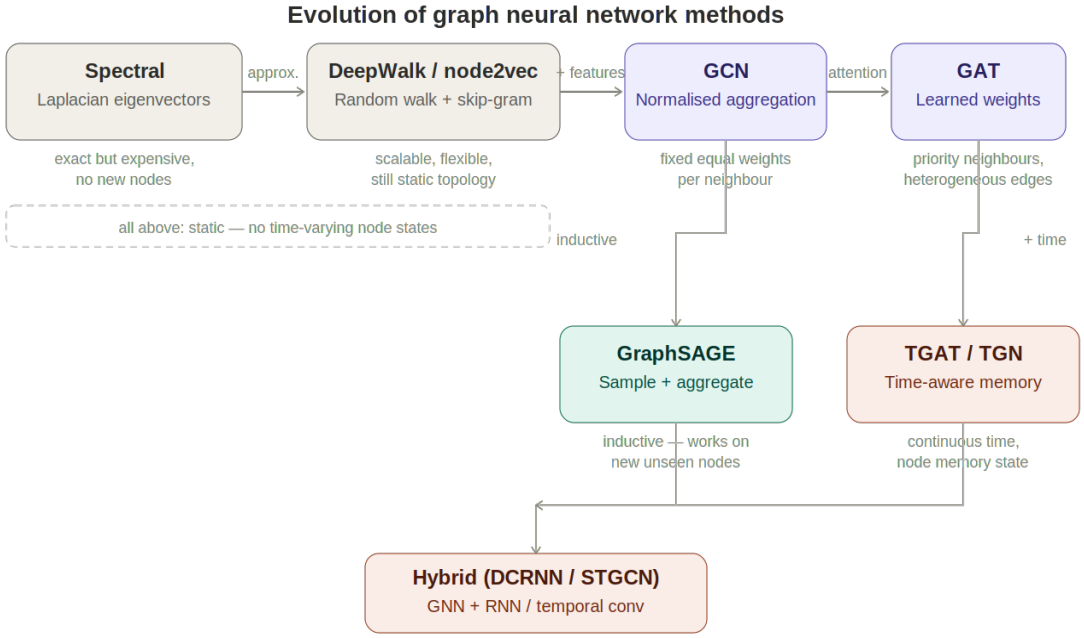

Figure A.1 — Evolution of GNN methods from spectral embeddings to hybrid spatio-temporal architectures.

A.2 Spectral Methods and the Graph Laplacian

The earliest approach to learning node representations was rooted in linear algebra. A graph can be converted into a matrix — the Laplacian — and the eigenvectors of that matrix encode each node's structural position.

A.2.1 From graph to matrix

For a graph with n nodes, the adjacency matrix A records which nodes are connected: A[i,j] = 1 if an edge exists between nodes i and j, and 0 otherwise. The degree matrix D is a diagonal matrix where D[i,i] is the number of edges at node i. The graph Laplacian is then:

```
L = D − A
```

The diagonal of L contains each node's degree. The off-diagonal entries are −1 wherever two nodes are connected and 0 elsewhere. This structure means L encodes both connectivity (who is linked to whom) and centrality (how many links each node has) in a single matrix.

A.2.2 The Cushing subnetwork

Consider a five-node subgraph: Permian Basin (node 1), Gulf Coast (node 2), Cushing terminal (node 3), a refinery (node 4), and a storage facility (node 5). Cushing is connected to all four other nodes; the other nodes are connected only to Cushing or to one other node.

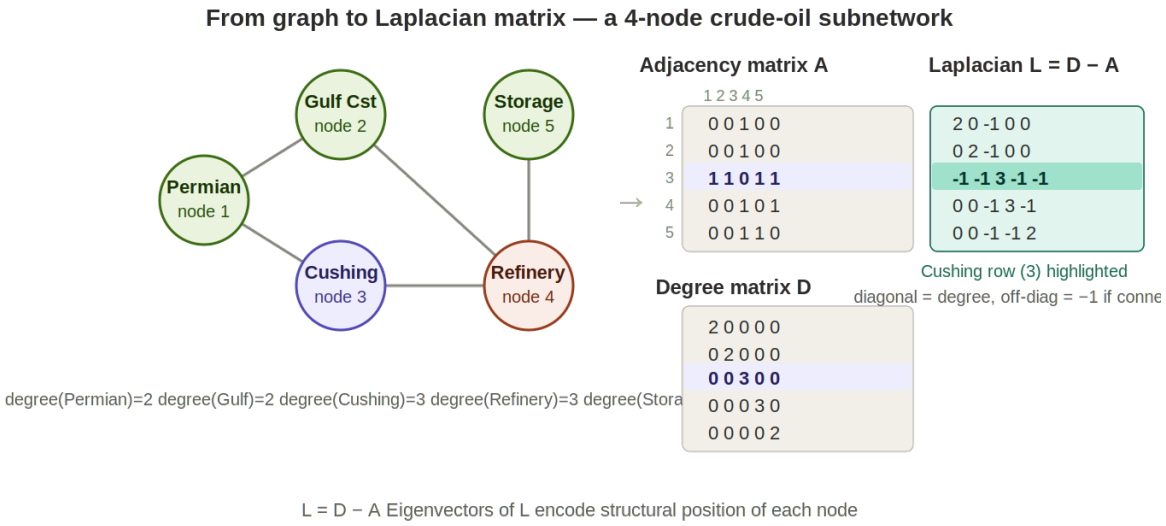

Figure A.2 — The five-node crude-oil subnetwork and its Laplacian matrix. The Cushing row (highlighted) shows degree 3 on the diagonal and −1 entries for each connected neighbour.

The eigenvectors of L can be computed by solving det(L − λI) = 0 for the eigenvalues λ, then substituting each λ back to find the corresponding vector. The eigenvector for the smallest non-zero eigenvalue — the Fiedler vector — assigns values to nodes such that structurally similar nodes (in the same cluster) receive similar values, and dissimilar nodes receive different values. Splitting the Fiedler vector at zero recovers the graph's primary structural partition without trying every possible grouping.

The limitation of spectral methods is that eigenvectors must be recomputed from scratch whenever the graph changes. Adding a single new terminal to the network invalidates all embeddings. This motivated a fundamentally different approach.

A.3 DeepWalk and node2vec — Random Walk Embeddings

DeepWalk (Perozzi et al., 2014) borrowed the key insight of word2vec from natural language processing: just as words that appear near each other in sentences tend to be semantically related, nodes that appear near each other in random walks tend to be structurally related.

A random walk starts at a node, hops to a randomly chosen neighbour, hops again from there, and repeats for a fixed number of steps. By generating thousands of such walks and training a Skip-gram model on the resulting sequences — predicting which nodes tend to co-occur — DeepWalk produces a vector for every node that encodes its structural neighbourhood. Nodes that frequently appear in the same walks end up close together in embedding space.

node2vec (Grover and Leskovec, 2016) extended this by introducing two hyperparameters that control whether walks favour exploring outward (capturing broad community structure, analogous to the Fiedler vector) or revisiting recent nodes (capturing tight local structure). Unlike spectral methods, neither approach requires eigenvector decomposition and both scale to very large networks.

The shared limitation is that these embeddings are derived purely from topology and do not incorporate the operational state of nodes — inventory levels, flow rates, throughput. A node's embedding does not change when its inventory changes. This motivated graph neural networks.

A.4 Neural Message Passing — the GNN Mechanism

Graph neural networks (GNNs) learn node representations through iterative neighbourhood aggregation. Each node starts with its own feature vector — in the crude-oil network, a vector of operational variables — and repeatedly updates that representation by pulling in information from its neighbours. Gilmer et al. (2017) showed that GCN, GAT, GraphSAGE and other architectures are all special cases of the same three-step mechanism, applied layer by layer.

A.4.1 The three steps

- **Send —** each node sends a message to its neighbours. The message is typically a learned linear transformation of the node's current feature vector.
- **Aggregate —** each node collects all incoming messages and combines them — usually by summing or averaging — into a single neighbourhood summary vector.
- **Update —** each node combines the neighbourhood summary with its own current representation to produce a new, updated embedding.

After one round, each node knows about its immediate neighbours. After two rounds, it knows about nodes two hops away. After k rounds, information has propagated across k-hop neighbourhoods — exactly as a physical disruption propagates through a pipeline network.

A.4.2 The Cushing example

To make this concrete, consider Cushing terminal connected to three assets with the following feature vectors (all flows in barrels per day):

Variable	Permian (node 1)	Gulf Coast (node 2)	Cushing (node 3)	Refinery (node 4)
Production (b/d)	800	600	0	0
Inflow (b/d)	0	0	1,400	950
Inventory (bbls)	0	0	45,000	0

Outflow (b/d)	800	600	1,350	950
Utilisation	0.82	0.71	0.68	0.91

Cushing holds 45,000 barrels in stock, receiving 1,400 b/d from the two pipelines and sending 950 b/d to the refinery — a slow net inventory build of 50 b/d.

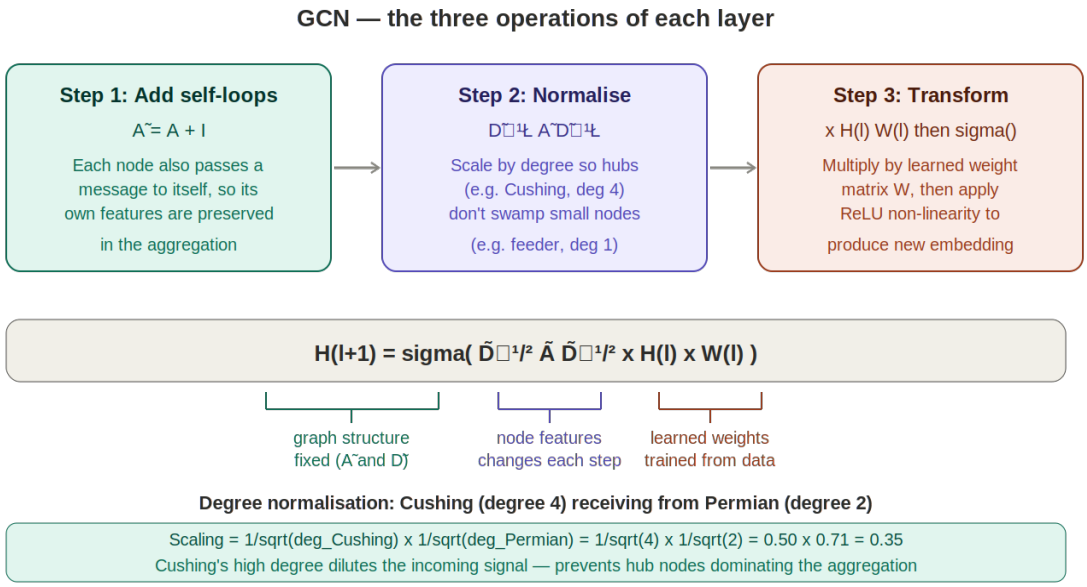

Figure A.3 — Round 1 of message passing at Cushing. Each neighbour sends a transformed feature vector; Cushing averages them and updates its own 64-dimensional representation.

In Round 1, each neighbour sends a message — a learned transformation W × feature_vector — to Cushing. Cushing averages the three messages and combines the result with its own features:

```
New representation = ReLU( W_self × [0, 1400, 45000, 1350, 0.68]
                         + W_neigh × avg(msg_Permian, msg_Gulf,
msg_Refinery) )
```

The weight matrices W_self and W_neigh are learned during training. After seeing many weeks of historical data, the model learns which variables from neighbouring nodes are most predictive of Cushing's future inventory — for example, that high Permian and Gulf Coast production rates predict rising inflows, and that a high refinery utilisation rate predicts rising outflows.

In Round 2, Cushing's updated Round 1 representation is passed back to its neighbours, which update their own embeddings accordingly. Cushing's Round 2 update then incorporates those — so it now indirectly knows about nodes two hops away: the production fields feeding the Permian pipeline and the distribution terminals receiving refined products from the Illinois refinery.

A.5 Graph Convolutional Networks (GCN)

GCN (Kipf and Welling, 2017) is a mathematically principled formalisation of message passing, derived from spectral theory via two simplifications.

A.5.1 From spectral convolution to GCN

Spectral graph convolution applies a filter to node features in the eigenvector space of the Laplacian: y = U g(Λ) Uᵀ x. Computing this exactly requires eigenvector decomposition. Hammond et al. (2011) proposed approximating g(Λ) using Chebyshev polynomials, which can be computed recursively from the Laplacian without eigenvectors. With K=1 (first-order only), the approximation becomes equivalent to aggregating information from the immediate neighbourhood. Kipf and Welling then tied the two remaining parameters and applied a renormalisation trick — adding self-loops to the adjacency matrix — to arrive at the GCN layer formula.

A.5.2 The GCN layer formula

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)} \right)$$

where $\tilde{A} = A + I$ is the adjacency matrix with self-loops added, $\tilde{D}$ is the corresponding degree matrix, $H^{(l)}$ is the matrix of node representations at layer l, $W^{(l)}$ is a learned weight matrix, and σ is a non-linear activation function (ReLU).

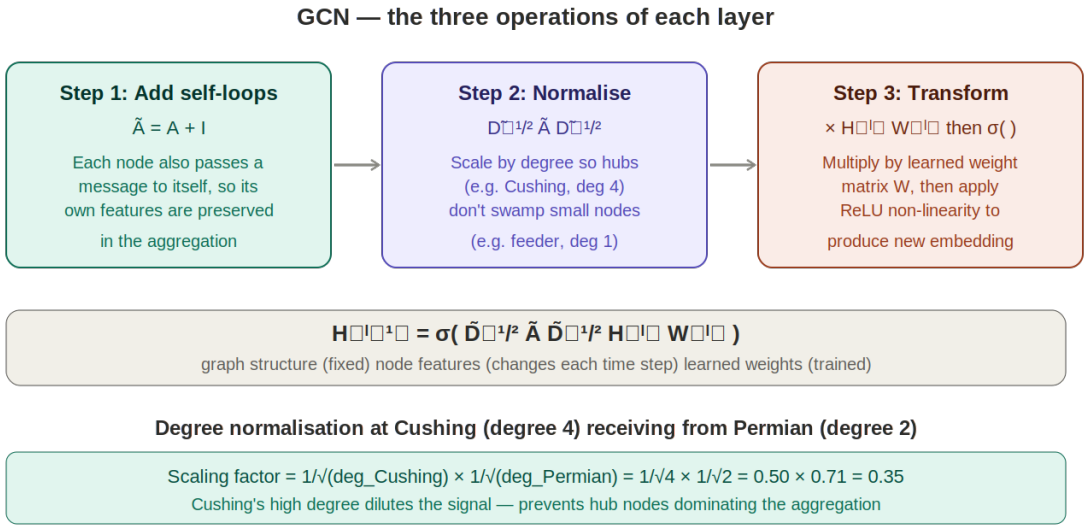

Figure A.4 — The three operations of a GCN layer: add self-loops, normalise by degree, apply learned linear transformation. The degree normalisation example shows Cushing (degree 4) receiving from the Permian node (degree 2) with a scaling factor of 0.35.

The degree normalisation is critical for logistics networks. Cushing as a major hub has many more connections than a small inland feeder terminal. Without normalisation, Cushing's messages would dominate the aggregation at every node it touches. Scaling by 1/√degree ensures that high-degree nodes do not swamp low-degree ones, making the GNN well-behaved across nodes with very different connectivity levels.

A.6 Attention and Inductive Learning — GAT and GraphSAGE

GCN applies the same aggregation weight to all neighbours. Two extensions address this limitation.

GAT (Veličković et al., 2018) replaces fixed degree-normalised weights with learned attention coefficients. For each pair of connected nodes, the model learns a scalar attention weight that reflects how informative that neighbour's state is for the target node. In the crude-oil network, GAT can learn to attend more strongly to a high-capacity pipeline than to a minor feeder line, or to weight the refinery's throughput more heavily than a distant storage node when predicting Cushing's near-term outflows.

GraphSAGE (Hamilton et al., 2017) learns the aggregation function itself rather than applying a fixed formula. Its key property is inductiveness: it can generate embeddings for nodes not seen during training. In an asset-centric network designed to grow — new terminals commissioned, new vessel classes added — inductiveness means new assets can be embedded immediately by aggregating from their neighbours, without retraining the entire model.

A.7 Temporal Extensions — TGAT, TGN, and Hybrid Architectures

All methods discussed so far operate on a static graph snapshot. For inventory forecasting, the time dimension is essential — Cushing's inventory at week t+1 depends on the trajectory of flows over the preceding weeks, not just the current snapshot.

TGAT (Xu et al., 2020) incorporates time encodings into the attention mechanism, allowing the model to weight recent interactions more heavily than older ones. TGN (Rossi et al., 2020) maintains an updatable memory state for each node, capturing cumulative flow history — a vessel node's memory encodes its cargo history across voyages, a terminal's memory encodes its seasonal loading patterns.

The hybrid architectures — DCRNN (Li et al., 2018) and STGCN (Yu et al., 2018) — combine a spatial GNN with a temporal sequence model. DCRNN models spatial propagation as directed diffusion (physically appropriate for pipeline flows) and uses a GRU encoder-decoder for multi-step forecasting. STGCN interleaves graph convolution with dilated temporal convolution, achieving faster training while capturing both short-term fluctuations and medium-term seasonal patterns in inventory levels.

These hybrid architectures are the primary model family candidates for the flow forecasting framework developed in this thesis, where the stable-topology asset-centric graph and weekly EIA data snapshots make the discrete-time snapshot approach directly applicable.

References

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. (2017). Neural Message Passing for Quantum Chemistry. ICML.

Grover, A. and Leskovec, J. (2016). node2vec: Scalable Feature Learning for Networks. KDD.

Hamilton, W. L., Ying, Z., and Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. NeurIPS.

Kipf, T. N. and Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR.

Li, Y., Yu, R., Shahabi, C., and Liu, Y. (2018). Diffusion Convolutional Recurrent Neural Network. ICLR.

Perozzi, B., Al-Rfou, R., and Skiena, S. (2014). DeepWalk: Online Learning of Social Representations. KDD.

Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Bronstein, M., and Monti, F. (2020). Temporal Graph Networks. ICML Workshop.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph Attention Networks. ICLR.

Xu, D., Rong, Y., Huang, J., Zhu, W., and Leskovec, J. (2020). Inductive Representation Learning on Temporal Graphs. ICLR.

Yu, B., Yin, H., and Zhu, Z. (2018). Spatio-Temporal Graph Convolutional Networks. IJCAI.